

```
/******
```

Welcome to GDB Online.

GDB online is an online compiler and debugger tool for C, C++, Python, Java, PHP, Ruby, Perl, C#, OCaml, VB, Swift, Pascal, Fortran, Haskell, Objective-C, Assembly, HTML, CSS, JS, SQLite, Prolog.

Code, Compile, Run and Debug online from anywhere in world.

```
*****/
```

```
import java.util.*;
public class Main
{
    public static void main(String[] args) {
        // Hashsets -> Homogenous Collection of unique elements (Non-sequential ->
insertion order is not maintained)
        // Real life applications - email ids, aadhar number, contacts

        // Syntax
        //HashSet<ClassType> hs = new HashSet<ClassType>();
        //int -> Integer

        //HashSet<Integer> hs = new HashSet<Integer>();
        //System.out.println(hs);

        // Basic Operations
        // add
        //hs.add(3);
        //hs.add(-2);
        //hs.add(-2);
        //hs.add(1);
        //hs.add(100);
        //hs.add(13);
        //hs.add(29);

        //System.out.println(hs);

        /// contains -> boolean
        //System.out.println(hs.contains(100));
        //System.out.println(hs.contains(-100));

        /// size
        //System.out.println(hs.size());

        // remove -> hs.remove(val) -> true/false
```

```

//System.out.println(hs.remove(-100));
//System.out.println(hs);

// print - using for each loop
//for(int val: hs){
//    System.out.println(val);
//}

// ArrayList vs hashset
//- Duplicates allowed vs no duplicates
//- Sequential vs Non-sequential

// Challenge 1 - Given an integer array as input to a function, add its elements to a
//hashset and return the hashset
// int[] arr = {-2, 1, 3, -2, 3, 100, 13, 29};
// System.out.println(convertToHashSet(arr));

// Challenge 2 - Given 2 hashsets as input, print their common elements
// HS1 = {0,-2,3,10}
// HS2 = {1,-2,3,4,5}
// Output = -2 3

// int[] arr = {0,-2,3,10};
// HashSet<Integer> hs1 = convertToHashSet(arr);
// int[] arr1 = {1,-2,3,4,5};
// HashSet<Integer> hs2 = convertToHashSet(arr1);

// commonHashSet(hs1, hs2);

// HashMap -> Homogeneous DS used to store key-value pair
// Real life applications -> Contacts, Email ID and Password, Dictionary
// Constructed from an HashSet itself, for every key there's a corresponding value
// Population of different states
// hm = {
//    "Delhi": 10,
//    "Punjab": 20,
//    "UP": 25,
//    "Rajasthan": 20,
//    "Goa": 5
// }

// Syntax

```

```

// HashMap<KeyClassType, ValueClassType> hm = new HashMap<KeyClassType,
ValueClassType>();
// HashMap<String, Integer> hm = new HashMap<String, Integer>();
// System.out.println(hm);

// Basic Operations

// add
// hm.put("Delhi", 10);
// hm.put("Punjab", 20);
// hm.put("UP", 25);
// hm.put("Rajasthan", 20);
// hm.put("Goa", 5);

// // update
// hm.put("Delhi", 5);

// System.out.println(hm);

// containsKey
// System.out.println(hm.containsKey("Delhi"));
// System.out.println(hm.containsKey("Bihar"));

// get
// System.out.println(hm.get("Delhi"));
// System.out.println(hm.get("Bihar"));

// size
// System.out.println(hm.size());

// remove
// System.out.println(hm.remove("Delhi"));
// System.out.println(hm.remove("Bihar"));
// System.out.println(hm);

// print -> for each loop -> keySet() to get keys of Hashmap into an Hashset
// System.out.println(hm.keySet());
// for(String key: hm.keySet()){
//     System.out.println(key + ":" + hm.get(key));
// }

// Challenge 3 - Given an integer array as input, return the corresponding frequency map
// Input: arr = [1, 4, 3, -2, 1, 1, 4, 5, 3]
// Output:

```

```
// hm = {  
// 1: 3,  
// 4: 2,  
// 3: 2,  
// -2: 1,  
// 5: 1  
// }
```

```
// hm = {  
// 1: 1,  
// 4: 1  
// }
```

```
// int[] arr = {1, 4, 3, -2, 1, 1, 4, 5, 3};  
// System.out.println(frequencyCount(arr));
```

```
HashMap<Object, Object> hm = new HashMap<Object, Object>();  
hm.put(null, null);  
System.out.println(hm);
```

```
}
```

```
public static HashSet<Integer> convertToHashSet(int[] arr){  
    HashSet<Integer> hs = new HashSet<Integer>();  
    for(int val: arr){  
        hs.add(val);  
    }  
    return hs;  
}
```

```
public static void commonHashSet(HashSet<Integer> hs1, HashSet<Integer> hs2){  
    for(int val: hs1){  
        if(hs2.contains(val)) System.out.print(val+ " ");  
    }  
}
```

```
public static HashMap<Integer, Integer> frequencyCount(int[] arr){  
    HashMap<Integer, Integer> hm = new HashMap<Integer, Integer>();  
    for(int val: arr){  
        if(hm.containsKey(val)) {  
            int existing_cnt = hm.get(val);  
            int updated_cnt = existing_cnt+1;
```

```
        hm.put(val, updated_cnt);
    }
    else hm.put(val, 1);
}
return hm;
}
}
```